Enabling Branching Support in Server-Side Vitruvius

Bachelor Thesis Proposal of

Linh Nguyen Phuong

At the KIT Department of Informatics
KASTEL – Institute of Information Security and Dependability

First examiner:	Prof. Dr.-Ing. Anne Koziolek
Second examiner:	Prof. Dr. Ralf Reussner
First advisor:	Dr. Raziyeh Dehghani
Second advisor:	Dr. rer. nat Arne Lange
Third advisor:	M.Sc. Martin Armbruster

01.12.2025 – 26.01.2026

Karlsruher Institut für Technologie
Fakultät für Informatik
Postfach 6980
76128 Karlsruhe

Contents

1 Motivation **1**
1.1 Introduction . . . 1
1.2 Goal of thesis . . . 2
1.3 Outline of this Proposal . . . 2

2 Foundation / Background **3**
2.1 Model-Driven Software Development . . . 3
2.2 View-Based Model-Driven Software Development . . . 3
2.2.1 Models, Metamodels, and View Integration . . . 3
2.2.2 Deltas and Consistency Preservation . . . 4
2.2.3 Framework Architecture and Integration Patterns . . . 4
2.3 Versioning and Branching in Model-Driven Software Development . . . 4
2.3.1 Foundations of Model Versioning in MDSD . . . 5
2.3.2 State-Based Versioning Approaches . . . 5
2.3.3 Operation-Based Versioning Approaches . . . 5
2.3.4 Core Activities in Model Versioning . . . 6
2.3.5 Branching in Model-Driven Software Development . . . 6
2.3.6 Requirements of Model Versioning and Branching Strategy . . . 7

3 Related Work **9**
3.1 Model Comparison and Versioning Frameworks . . . 9
3.2 Pointer-Based File Storage: Git . . . 11

4 Conception **13**
4.1 Requirements for Integrating Version Control and Branching into Vitruvius 13
4.1.1 General Model Versioning Requirements . . . 13
4.1.2 Vitruvius-specific requirements . . . 14
4.2 Planned Approach . . . 21
4.2.1 Branch Management Design . . . 21
4.2.2 Delta Computation and Storage Concept . . . 22
4.2.3 Merge Infrastructure . . . 24
4.2.4 Integration Architecture: Git and Vitruvius Interaction . . . 25
4.3 Expected Workflow . . . 26

5 Organisation **29**
5.1 Phases . . . 29
5.1.1 Conception and Design . . . 29

5.1.2 Implementation Strategy 29
5.1.3 Validation and Evaluation 30
5.2 Timeline 31

Bibliography **33**

1 Motivation

This chapter introduces the background and motivation of the thesis by outlining the challenges of collaborative model-driven development and explaining why the lack of branching support in the Vitruvius server limits its applicability.

1.1 Introduction

Modern software development increasingly relies on model-driven engineering (MDE) techniques, where software systems are represented and managed through multiple interrelated models. These models capture different viewpoints, abstraction levels, and architectural perspectives. Ensuring their consistency during evolution is challenging, particularly in collaborative environments where multiple developers may work in parallel. Vitruvius, a framework for view-based software development, addresses this challenge by providing an infrastructure that treats a set of heterogeneous models as a unified Virtual Single Underlying Model (V-SUM)[12]. Through this abstraction, developers can work directly on the modeling views that are most appropriate for their tasks, while the framework ensures that the different models remain synchronized, coherent, and consistent. This is achieved through a combination of declarative and imperative consistency preservation rules that propagate changes across views as deltas, enabling a flexible and extensible approach to multi-view model management.

The current Vitruvius server, though sophisticated in delta propagation and consistency management, operates on a single linear evolution of the V-SUM. All changes are applied sequentially to this global state, preventing multiple independent development lines. In collaborative or large-scale modeling environments, this approach limits experimentation, parallel feature development, and concurrent work, reducing the framework's scalability and applicability

Branching, a fundamental concept in modern version control systems, allows multiple isolated workstreams to evolve independently. Native server-side branching in Vitruvius would enable developers to manage divergent versions of the V-SUM safely, supporting parallel work while preserving consistency. Branching also lays the foundation for essential versioning operations such as merging, conflict detection, and conflict resolution.

1.2 Goal of thesis

This thesis aims to close this gap by designing and implementing server-side branching support for Vitruvius, focusing on:

- Explicit branch management (creation, switching, deletion, and tracking parent-child relationships)
- Efficient delta computation and storage per branch
- Safe merging of divergent model versions with conflict handling

By extending Vitruvius in this way, the framework can support collaborative, scalable, and parallel model-driven development, bridging the gap between its strong consistency mechanisms and the practical requirements of multi-developer software engineering workflows.

1.3 Outline of this Proposal

This proposal proceeds as follows. First, we summarize the conceptual foundations relevant to server-side branching in model-driven development, including multi-view consistency, delta-based change propagation, and versioning principles. Next, we analyze existing branching and model versioning technologies, such as EMF Compare, AMOR, and other model versioning frameworks to identify which techniques, concepts, or components may be reused or adapted within Vitruvius. Based on this comparison, we derive core requirements for branching within the Vitruvius context. Finally, we outline the planned design and implementation approach, describing how these requirements will influence architectural decisions and how the new branching functionality integrates with the existing Vitruvius server infrastructure.

2 Foundation / Background

To understand the problem domain and the technical context of the thesis, the relevant conceptual and technical foundations will be introduced, including model-driven software development, view-based consistency management, and model versioning and branching approaches.

2.1 Model-Driven Software Development

Model-Driven Software Development (MDSD) treats models as primary artifacts throughout the software development process. Models capture different system aspects at various abstraction levels, enabling automated transformations, code generation, and early validation of design properties. By raising the abstraction level, MDSD supports more systematic reasoning about system behavior and improves communication among stakeholders. At the same time, the use of multiple heterogeneous models introduces challenges: different views may evolve independently, inconsistencies can arise easily, and maintaining semantic coherence requires dedicated mechanisms for synchronization and coordinated evolution. Addressing these challenges is essential for applying MDSD effectively in collaborative development settings[2].

2.2 View-Based Model-Driven Software Development

2.2.1 Models, Metamodels, and View Integration

View-based development decomposes a complex system into multiple models, each representing a distinct concern such as structure, behavior, deployment, or performance. While these models focus on different aspects, they frequently overlap, sharing elements that must remain consistent across views. General frameworks address this by conceptually integrating models into a coherent structure, allowing developers to edit each view independently while maintaining global system semantics[6].

Vitruvius exemplifies this approach through the *Virtual Single Underlying Model (V-SUM)[12]*, which provides a logical unification of all system models. Developers interact with task-specific views, while the V-SUM ensures that underlying models remain synchronized. This abstraction hides the complexity of cross-view consistency and provides a foundation for supporting incremental changes and collaborative modeling.

2.2.2 Deltas and Consistency Preservation

Changes in a model are typically represented as *deltas*, capturing additions, deletions, or modifications relative to a previous state. Delta-based processing allows frameworks to propagate changes incrementally, updating only affected parts of the model and reducing computational overhead[11]. Consistency preservation ensures that interdependent models remain semantically coherent. This is generally achieved through declarative rules, which define invariants that must hold across models, and imperative rules, which determine how changes should propagate to satisfy these invariants.

In Vitruvius, every change in a view generates a delta, which is propagated to other views according to both declarative and imperative rules[12]. Correspondence models explicitly link related elements across views, enabling precise and efficient updates. This combination of deltas and consistency rules allows incremental evolution while guaranteeing that all views remain semantically consistent.

2.2.3 Framework Architecture and Integration Patterns

View-based frameworks generally separate three key concerns:

- View definitions, which provide task-specific abstractions of the system.
- Change management, which provide task-specific abstractions of the system.
- Consistency preservation, which enforces invariants and synchronizes related model elements.

Vitruvius implements these principles through its modular architecture: views provide projections of the V-SUM, delta propagation manages updates, and correspondence models maintain element links across views[12]. This separation of concerns ensures extensibility and flexibility. Furthermore, the modular design creates natural extension points for additional features such as server-side branching, where each branch could maintain an independent V-SUM, delta history, and correspondence state while reusing the existing view-based editing and consistency mechanisms.

2.3 Versioning and Branching in Model-Driven Software Development

Versioning is a fundamental activity in modern software development, enabling teams to track evolution, manage parallel development, and ensure the recoverability of artifacts. While established textual version control systems such as Git operate on line-based representations, these techniques cannot be directly transferred to MDSD. Models are structured, typed graphs that must conform to metamodels and frequently include cross-references and well-formedness constraints[16]. As a consequence, dedicated model versioning techniques

have emerged to address the structural and semantic complexity inherent in model artifacts. This section introduces foundational concepts of model versioning in MDSD, outlines established versioning approaches as identified in the literature, and discusses the general requirements that influence the suitability of a versioning strategy. These insights form the basis for evaluating an appropriate design for server-side model versioning and branching in the Vitruvius framework.

2.3.1 Foundations of Model Versioning in MDSD

Model versioning refers to the processes, algorithms, and representations used to capture, compare, and integrate different versions of models. In contrast to textual artifacts, model changes cannot easily be expressed through line-based differences, as the order of elements in serialized formats such as XMI is not semantically meaningful[15]. Instead, model versioning techniques must operate on the structural and semantic properties of models defined by their metamodels[4]. Two central challenges arise. First, identifying corresponding elements across versions requires robust matching strategies, typically relying on unique identifiers, name heuristics, or structural similarities. Second, model versioning must preserve the well-formedness and consistency of models throughout the versioning lifecycle. This includes respecting metamodel constraints, handling cross-model references, and avoiding the introduction of invalid model states during and also after merging[4].

2.3.2 State-Based Versioning Approaches

State-based versioning approaches operate by comparing two snapshots of a model to compute their differences. The comparison process usually involves a matching phase that aligns corresponding model elements, followed by a differencing phase that identifies added, removed, or modified elements. A representative framework is EMF Compare[5], which provides two-way and three-way diff and merge capabilities for EMF-based models. State-based approaches offer several advantages: they can be applied independently of the modeling editor, and they often provide mature user interfaces for inspecting and resolving differences. Their primary limitations arise from computational cost, as matching and differencing may be expensive for large or deeply nested models. Moreover, state-based approaches typically operate at a syntactic level, meaning that higher-level, semantic changes (e.g., refactorings or composite operations) may not be reflected accurately[4].

2.3.3 Operation-Based Versioning Approaches

Operation-based versioning approaches record the editing actions performed during model evolution. Instead of comparing full model states, these approaches capture fine-grained change operations such as creating, deleting, or modifying elements. EMFStore is a prominent example, maintaining an operation log that can be replayed or merged to derive new

model versions[13]. Operation-based versioning offers several benefits. Deltas remain compact, as only the operations themselves are stored, expensive diff computations are avoided, and the recorded operations often reflect the developer's intent more accurately. However, these systems may become dependent on specific modeling editors or interaction patterns, since the recorded operations must correspond to the semantics of the underlying editor. Handling composite operations is also challenging: if a user-level action spans multiple low-level operations, it must be represented atomically to avoid invalid or inconsistent intermediate states during merging[4].

2.3.4 Core Activities in Model Versioning

Independent of whether a versioning approach is state-based or operation-based, the versioning process is often described as consisting of these fundamental activities[4]:

- **Operation detection**: determining operations applied on model versions, either as atomic operation or as a composite of multiple atomic operations.
- **Matching**: identifying corresponding model elements across versions using identifiers or structural heuristics.
- **Differencing**: determining changes between versions, either as low-level operations or as state-based modifications.
- **Conflict Detection**: identifying situations in which concurrent changes cannot be combined without violating model consistency.
- **Conflict Resolution**: providing automated or manual strategies for selecting which changes should take precedence.
- **Merging**: integrating changes into a unified model version, typically using three-way merge strategies.

While syntactic conflicts are commonly addressed, semantic conflict detection remains a limited capability and is still regarded as an open research challenge[6]. Only a small number of approaches, such as SMOVER or domain-specific conflict detectors, address semantic inconsistencies explicitly.

2.3.5 Branching in Model-Driven Software Development

Branching enables parallel development by allowing multiple, isolated sequences of changes to coexist until they are eventually integrated. In textual version control, branching is a well-established concept supported by dedicated primitives such as *branch creation*, *switching*, and *merging*. However, explicit branching support is largely absent or rudimentary in most existing model versioning systems. Several reasons might contribute to this gap. First, model merges typically rely on complex matching and differencing, making repeated merges across multiple diverging branches costly. Second, model semantics and cross-references introduce

additional constraints that can lead to conflicts that do not exist in textual artifacts. Third, most versioning systems in the MDSD ecosystem assume linear or centralized repositories with limited support for long-lived or parallel branches. These challenges are magnified in the context of view-based model development. When multiple views represent different projections of a holistic system, branching must ensure that cross-view consistency is preserved across all branches. Versioning must therefore operate not only on isolated model states but on interdependent model deltas and synchronization rules[15].

2.3.6 Requirements of Model Versioning and Branching Strategy

To evaluate the suitability of a model versioning approach-independently of a specific framework, several general features must be considered: accuracy of change representation, conflict detection, consistency preservation, scalability and performance,... These general requirements not only serve as general criteria for evaluating model versioning approaches but also form the foundation for Vitruvius-specific requirement discussed later. In Chapter **4.1**, they are integrated directly into the structured requirements model for the Vitruvius branching extension.

3 Related Work

The design of branching support for Vitruvius requires evaluation of existing model versioning approaches. This chapter examines state-of-the-art frameworks for model comparison, versioning, and conflict detection, analyzing their suitability for integration with Vitruvius's consistency-preservation architecture. Each approach is assessed based on compatibility with Vitruvius's operation-based change tracking, architectural overhead, and support for branching semantics.

3.1 Model Comparison and Versioning Frameworks

EMF Compare EMF Compare is the standard Eclipse solution for state-based model comparison, identifying differences between model snapshots via matching, difference detection, and merging. Its advantages lie in its mature matching algorithms, which effectively cope with element renamings and structural modifications, and in its seamless integration with the Eclipse ecosystem[19]. Nonetheless, EMF Compare relies solely on state-based comparison and has no access to the underlying operation sequences. This leads to key drawbacks for Vitruvius: the semantics of operations are discarded, meaning that intermediate modifications that ultimately revert to the original state remain undetected, even though they may have triggered consistency rules; causal relationships are not captured, making it impossible to distinguish direct user edits from automatically propagated ones; and fine-grained delta sequences cannot be recovered, which rules out replay and rollback functionalities. Vitruvius already offers comparable state-based comparison with the StateBasedChangeResolutionStrategy, which operates on Vitruvius's EChange metamodel. Instead of adding EMF Compare as an external dependency, this work enhances the existing strategy to support branch-sensitive delta computation while reusing established matching heuristics.

EMF Diff/Merge EMF Diff/Merge extends EMF Compare with enhanced three-way merging capabilities, using a specilized algorithm to compare models against a common base. It supports both two-way and three-way comparison and provides extensible mechanisms for detecting and resolving conflicts at the model element level. DiffMerge operates on model states and focuses on computing differences between complete models rather than tracking their evolution over time[4]. Consequently, it does not provide facilities for version history management, branch representation, or delta persistence. Despite these limitations, EMF Diff/Merge offers reusable mechanisms for state-based comparison and conflict detection, which may complement operation-based merging approaches.

CDO CDO is a client-server framework offering persistent storage, transactions, and native branching, with strengths in scalability, transactional guarantees, and branch management with temporal queries[15]. However, it conflicts with Vitruvius's architecture: CDO requires all models in its repository, replacing V-SUM, the core abstraction for consistency. Its persistence model prioritizes query performance over the atomic EChange sequences Vitruvius relies on, and integrating CDO would complicate consistency preservation and require re-engineering the ChangePropagationSpecification. Adoption also entails significant overhead, including migrating from XMI to databases, implementing metamodel adapters, and maintaining server infrastructure - a full framework reimplementation rather than a simple extension. Seybold et al.[17] note CDO's steep learning curve and limited support for model versioning and co-evolution.

EMFStore EMFStore[13] implements operation-based versioning, capturing modifications as sequences of operations instead of full state snapshots. Although this operation-centric paradigm is conceptually similar to Vitruvius's change tracking, EMFStore, similar to CDO, demands replacing the existing repository infrastructure, and its operation model is incompatible with Vitruvius's EChange metamodel, making translation layers necessary. In addition, EMFStore has been declared end-of-life and is no longer maintained for current EMF versions.

EML The Epsilon Merging Language (EML) is a rule-based language for defining custom model merge strategies on top of EMF models. EML enables developers to specify how elements from two or more input models should be matched, merged, or discarded, and supports fine-grained, domain-specific conflict resolution logic. Its main strength lies in its expressiveness and flexibility, allowing merge behavior to be tailored precisely to specific modeling languages or project conventions[14][18]. However, EML operates purely on model states and assumes that input models are provided as complete snapshots. It does not offer facilities for version history management, branch tracking, or operation recording. Furthermore, EML requires manual specification of merge rules, which introduces additional modeling effort and shifts responsibility for correctness to the merge definition itself. In the context of Vitruvius, EML is therefore not suitable as a primary merge mechanism. Nevertheless, its rule-based merge concepts provide valuable inspiration for defining configurable and extensible conflict resolution strategies on top of Vitruvius's operation-based merge infrastructure.

AMOR AMOR represents a significant research contribution, introducing sophisticated conflict detection that combines operation-based and state-based analysis[4]. Its main contribution is a comprehensive conflict taxonomy defining atomic conflict patterns such as delete-delete, update-update, and delete-reference. In addition, AMOR supports composite operation detection, recognizing high-level refactorings from atomic change sequences to enable semantic conflict analysis. While the framework's theoretical foundations are sound, its implementation is no longer maintained and does not cover consistency-preservation-specific conflicts unique to Vitruvius. This work builds on AMOR's concepts: atomic conflict patterns form the basis of syntactic conflict detection, composite operation detection informs

future extension points, and conflict prioritization enables automatic resolution of trivial cases. These ideas are extended with Vitruvius-specific semantic conflicts to integrate fully with consistency-preservation rules.

Difference Metamodel Approach The Difference Metamodel approach from Cicchetti et al. [7] represents model differences via an explicit metamodel with first-class types for changes, enabling tool-independent analysis but introducing overhead in metamodel maintenance, serialization, and tool handling. Vitruvius already provides an explicit change metamodel, the EChange hierarchy, representing atomic operations. Instead of adding a separate difference metamodel, this work extends EChange with branch-aware metadata (branch UID, timestamp, author), achieving structured, explicit change representation while maintaining architectural consistency and avoiding the complexity of parallel metamodels.

Efficient Model Matching Similarity-based model matching constitutes the foundation of state-based differencing and merging approaches, but its computational complexity grows cubically with the number of model elements, making it a performance bottleneck for large models. He et al.[10] address this limitation by exploiting the observation that consecutive model versions typically modify only small portions of a model. Their approach accelerates similarity-based matching by identifying equivalent or highly similar subtrees, allowing unchanged regions to be matched efficiently using hash-based techniques. Experimental results show substantial performance improvements while often reducing matching errors. Although this work focuses on optimizing matching algorithms rather than version management or branching semantics, it highlights the importance of scalable model comparison as a prerequisite for efficient delta computation and merging. These insights underpin the design goals of this thesis, which assumes efficient matching mechanisms and focuses instead on integrating branch-aware versioning and merge infrastructure into Vitruvius.

3.2 Pointer-Based File Storage: Git

Git is a widely used distributed version control system for source code, providing a reference point for lightweight, pointer-based branching and efficient delta storage. Git stores all repository data in a local ‘.git/‘ directory using plain files rather than a database. Branches are represented as simple pointers to commits, and the current branch is tracked via a HEAD reference. Commits themselves store content objects (blobs, trees) and metadata such as author, timestamp, and parent pointers, while merges create new commits linking multiple parents. Git’s storage model relies on deltas and snapshots, using hashes for integrity and efficient retrieval, which enables rollback, branching, and merging without a central database[20][8]. This decentralized, file-based approach inspired the proposed ‘.vitruvius/‘ storage concept, where model versions, deltas, branch pointers, and snapshots are stored similarly, enabling branch-sensitive versioning and conflict detection while remaining compatible with Vitruvius’s operation-based change tracking.

4 Conception

This chapter establishes how branching and model versioning are realized in Vitruvius by defining the necessary requirements, designing the supporting subsystems, and explaining their interactions. It demonstrates how the proposed architecture enables parallel model evolution, ensures consistency across branches, and provides mechanisms for efficient change tracking, merging, and conflict resolution.

4.1 Requirements for Integrating Version Control and Branching into Vitruvius

To design a branching-enabled and version-aware Vitruvius server, a unified requirements model is introduced. This model consists of two complementary layers:

- General dimensions of model versioning identified in the literature, which inform the design choices but do not impose strict constraints on Vitruvius.
- Core system-level requirements directly derived from the thesis objectives

4.1.1 General Model Versioning Requirements

General requirements capture essential capabilities expected form any model-versioning or branching solution, independent of a specific framework or metamodel. These requirements define the conceptual expectations and serve as an evaluation criteria for assessing the suitability of versioning strategies.

- **Branching Support**: The approach must support diverging development lines, allow parallel change histories, and provide efficient merge mechanisms.
- **Accuracy of Change Representation**: The chosen approach must correctly capture fine-grained deltas, composite operations, and meaningful semantic changes to models.
- **Conflict Detection Strength**: The approach should support detecting both syntactic and semantic conflicts to ensure the correctness of merges and parallel development.
- **Consistency Preservation**: Throughout evolution and merging, the approach must maintain model integrity, including the preservation of inter-model and cross-view consistency.

- **Scalability and Performance**: The approach should efficiently handle large models, deep nested structures, and long variant histories without degrading responsiveness.
- **Language and Tool Independence**: The approach should operate across heterogeneous modeling languages, metamodels, and modeling environments.
- **Extensibility**: The approach should allow custom conflict handlers, merge strategies, and domain-specific semantic extensions.

4.1.2 Vitruvius-specific requirements

To extend the Vitruvius server with native support for branching and model versioning, the core requirements can be structured into three main functionality areas: (i) *branching management* (ii) *delta computation and storage*, (iii) *merging infrastructure*. These categories reflect the essential technical functions needed to enable parallel developement, maintain historical evolution data and support robust merging mechanism. The detailed, concrete requirements for each category will be specified in the subsequent sections.

Branch Management: Vitruvius leverages Git's native branching infrastructure while extending it with model-specific metadata and validation. Branches are managed through Git's standard operations (checkout, branch, merge), with Vitruvius hooks ensuring VSUM consistency at critical lifecycle points. Branch metadata including parent-child relationships, lifecycle states, and VSUM-specific configuration is maintained in the `.vitruvius/` directory, versioned alongside metamodels to ensure consistency across branch switches.

- **BR-1:** Create new branches from any existing branch.
  - Git operation: `git checkout -b <branch-name>`
  - Post-creation hook: `.git/hooks/post-checkout`
  - Extension point: `branch.BranchManager.initializeBranchMetadata()`
  - Storage: Git refs/ + `.vitruvius/branches/<branch-name>.metadata`
  - REST API: POST /branches/create (wraps Git command)
- **BR-2:** List all existing branches with their metadata.
  - Git operation: `git branch -list`
  - Extension point: `branch.BranchManager.listBranchesWithMetadata()`
  - Metadata loading: Parse `.vitruvius/branches*.metadata`
  - REST API: GET /branches
- **BR-3:** Switch between branches using either branch name or UID, loading the corresponding model state.
  - Git operation: `git checkout <branch-name>`

  - post-checkout hook: `.git/hooks/post-checkout`
  - Extension point: `branch.BranchManager.rebuildWorkspace()`
  - State reconstruction: Load metamodels from Git working directory, rebuild VSUM indexes
  - REST API: POST/branches/switch
- **BR-4:** Delete branches manually.
  - Git operation: `git branch -d <branch-name>`
  - Extension point: `branch.BranchManager.cleanupBranchMetadata()`
  - Cleanup: Remove `.vitruvius/branches/<branch-name>.metadata`
  - REST API: DELETE/branches/{name}
- **BR-5:** Automatically delete branches after successful merges.
  - Git operation: `git branch -d <branch-name>` (after merge)
  - Trigger: `.git/hooks/post-merge`
  - Extension point: `merge.MergeEngine.cleanupMergedBranch()`
- **BR-6:** Support both human-readable branch names and unique identifiers (UIDs).
  - Implementation: Git's native branch naming + commit SHA resolution
  - Extension point: `branch.BranchManager.resolveBranchIdentifier(String identifier)`
  - Supports: branch names (e.g., `feature/add-component`), commit SHAs, tags
- **BR-7:** Enable searching and finding branches by name or UID.
  - Git operation: `git branch -list <pattern>`
  - Extension point: `branch.BranchManager.findBranches(String pattern)`
  - REST API: GET /branches/search?q={query}
- **BR-8:** Track and store parent-child relationships between branches.
  - Git implementation: Commit graph (accessed via `git log -graph`)
  - Metadata storage: `.vitruvius/branches/<branch-name>.metadata`
  - Extension point: `branch.BranchManager.getBranchTopology()`
  - Parent reference: Stored in branch metadata, validated against Git commit graph
- **BR-9:** Maintain branch lifecycle states to support automated management and querying.
  - Storage: `.vitruvius/branches/<branch-name>.metadata`
  - Extension point: `branch.BranchState` Enum (ACTIVE, MERGED, DELETED)

  - State Transition: Enforced in `BranchManager` and Git hooks
  - Update triggers: pre-commit, post-merge, branch deletion
- **BR-10:** Prevent creating branches with duplicate names to avoid ambiguity.
  - Git validation: Git inherently prevents duplicate branch names
  - Extension point: `branch.BranchManager.validateBranchName()`
  - Additional validation: Check for naming conflicts in metadata

Delta Computation and Storage: The system adopts a hybrid storage architecture that delegates file-level versioning to Git while extending it with Vitruvius-specific semantic change tracking. Git manages metamodel files through its standard object database, providing compression, history tracking, and distributed synchronization. Vitruvius complements this with semantic changelogs stored in `.vitruvius/changelogs/` that capture operation-level changes essential for consistency validation and semantic conflict detection. All extension points listed below are located in the Vitruv main repository under `tools.vitruv.change.`, unless stated otherwise.

- **DE-1:** Compute deltas representing changes applied on each branch or since a common ancestor.
  - File-level delta: `git diff <commit1> <commit2>`
  - Semantic delta: `composite.branch.BranchDeltaComputer`
  - Extension point: `composite.branch.BranchDeltaComputer.computeSemanticDelta()`
  - Input: Loads models from Git commits using `git show <commit>:<file>`
  - REST API: GET /deltas?from={source}&to={target}
- **DE-2:** Efficiently store and retrieve delta sequences for comparison, merging, rollback, or reconstruction of model states.
  - File-level storage: Git object database (blobs, trees, commits)
  - Compression: Git packfiles with delta compression
  - Semantic changelogs: `.vitruvius/changelogs/<commit-sha>.changelog`
  - Extension point: `storage.SemanticChangelogManager`
  - Retrieval: Git operations + changelog parsing
- **DE-3:** Focus primarily on XMI-based models.
  - Storage format: XMI files versioned by Git
  - Canonical serialization: `storage.XMICanonicalizer`
  - Extension point: `storage.XMISnapshotManager`

  - Git handling: Standard text file versioning with custom merge drivers

- **DE-4:** Preserve the sequence of changes, including those triggered by composed/propagated operations, to allow rollback or replay of changes.
  - Commit sequence: Git commit history (`git log`)
  - Operation sequence: `.vitruvius/changelogs/<commit-sha>.changelog`
  - Extension point: `atomic.EChange` with branch-aware metadata
  - Replay mechanism: `storage.ChangeReplayer`
- **DE-5:** Enable developers to mark specific model states as versions by assigning a version ID.
  - Git operation: `git tag <version-id>`
  - Extension point: `tools.vitruv.framework.vsum.VersioningService.createVersion()`
  - Metadata: `.vitruvius/versions/<version-id>.metadata`
  - REST API: POST /branches/{id}/versions
- **DE-6:** Store metadata for each version, including version ID, timestamp, author and optional version description.
  - Git metadata: Commit message, author, timestamp (from `git log`)
  - Extended metadata: `.vitruvius/versions/<version-id>.metadata`
  - Extension point: `tools.vitruv.framework.vsum.model.Version` Entity
  - Storage: File-based in `.vitruvius/versions/`, versioned with Git
- **DE-7:** Support hybrid storage strategy combining full snapshots at developer-defined versions and incremental change logs between versions.
  - Snapshots: Git commits (full file trees)
  - Incremental storage: Git's internal delta compression in packfiles
  - Semantic layer: `storage.HybridDeltaStorage`
  - Optimization: Git automatically manages snapshot vs. delta storage
- **DE-8:** Support rollback to any previous version on a branch.
  - Git operation: `git reset -hard <commit>` or `git checkout <commit>`
  - Extension point: `tools.vitruv.framework.vsum.VersioningService.rollbackToVersion()`
  - Post-rollback: Rebuild VSUM workspace via post-checkout hook
  - REST API: POST /branches/{id}/rollback

Merge Infrastructure: A dedicated merge subsystem reconciles diverging branch histories through Git's three-way merge infrastructure, extended with Vitruvius-specific semantic merge drivers and consistency validation. Git handles merge orchestration and file-level conflict detection, while custom merge drivers registered for non-line-based files perform model-aware merging. Post-merge hooks validate VSUM consistency and detect semantic conflicts that cannot be identified through text-based comparison. All extension points listed below are located in the Vitruv main repository under `tools.vitruv.framework.`, unless stated otherwise.

- **MG-1:** Enable prioritizing commits during merge.
  - Git operation: Merge strategy selection (`git merge -s <strategy>`)
  - Extension point: `merge.ConflictPrioritizer`
  - Implementation: Custom merge driver logic
- **MG-2:** Detect and handle conflicts at different levels during merge:
  - File-level conflicts: Detected by Git merge
  - Custom merge drivers: `.git/config` merge driver registration
  - Extension point: `merge.ConflictDetector`
  - Implementation:
    * Syntactic conflicts (operation-level): `merge.syntactic.SyntacticConflictDetector`
    * Semantic conflicts (CPR violations): `merge.semantic.SemanticConflictDetector`
    * Invoked by: Custom merge drivers + post-merge hook
  - Conflict Representation: `merge.Conflict`
  - Conflict markers: Semantic conflict markers in merged files
- **MG-3:** Classify detected conflicts by priority level (HIGH, MEDIUM, LOW).
  - Extension point: `merge.ConflictPriority` Enum
  - Classification: `merge.ConflictPrioritizer.assignPriority()`
  - Stored in: Conflict metadata for resolution workflow
- **MG-4:** Support two-way merge which compares only source and target.
  - Git operation: `git diff <source> <target>`
  - Extension point: `merge.MergeEngine.twoWayMerge()`
  - Use case: Initial import or branch comparison without LCA
- **MG-5:** Support three-way merge using the lowest common ancestor.
  - Git operation: `git merge <branch>`

  - LCA Computation: `git merge-base <branch1> <branch2>`
  - Extension point: `merge.MergeEngine.threeWayMerge()`
  - Custom merge driver: Invoked for `.ecore`, `.reactions`.. files
  - Driver location: `vitruvius-merge` script registered in `.git/config`
  - REST API: POST /merges
- **MG-6:** Maintain the integrity of the overall model history during merges.
  - Git merge commits: Dual-parent commits created by `git merge`
  - Extension point: `merge.model.MergeCommit` Entity
  - Metadata: `.vitruvius/merges/<merge-commit-sha>.metadata`
  - Implementation:
    * Post-merge hook stores additional merge metadata
    * Tracks conflict resolutions and validation results
- **MG-7:** Apply non-conflicting changes automatically without user intervention.
  - Git auto-merge: Standard Git merge for non-conflicting files
  - Custom merge driver: Auto-merge for non-conflicting model changes
  - Extension point: `merge.MergeEngine.applyNonConflicting()`
  - Execution: Within custom merge driver before returning to Git
- **MG-8:** Support conflict resolution modes:
  - Extension point: `merge.resolution.ConflictResolver`
  - Implementation:
    * `merge.resolution.AutomaticResolver` (low-priority conflicts)
    * `merge.resolution.SemiAutomaticResolver` (medium-priority)
    * `merge.resolution.ManualResolver` (high-priority)
  - Integration: Invoked by custom merge drivers and post-merge hooks
  - UI: Conflict resolution interface for manual intervention
- **MG-9 (optional):** Implement conflict tolerance strategies, such as delayed merging or soft warnings for minor conflicts.
  - Extension point: `merge.MergeEngine.setConflictTolerance()`
  - Configuration: `.vitruvius/merge-config.yaml`
  - Warnings: Stored in merge metadata, displayed in UI

- **MG-10:** Validate the merged model state before finalizing the merge to ensure no integrity violations were introduced.
  - Git hook: `.git/hooks/post-merge`
  - Extension point: `merge.semantic.SemanticConflictDetector.validateMergedState()`
  - Integration: `ChangePropagationSpecification.executeAll()`
  - Timing: After applying all resolutions, before allowing commit
  - Failure handling: Abort merge, provide diagnostic report

User Interface The system provides a set of model-aware user interfaces that support delta inspection, version exploration, and conflict resolution. All interfaces operate on semantic model changes provided by Vitruvius.

- **UI-1:** Provide a user interface for visualizing model deltas.
  - Data source: Git-based version storage (e.g., `git diff`)
  - Semantic visualization: `changevisualization.SemanticDeltaVisualizer`
  - Extension point: `changevisualization.DeltaVisualizationService`
  - Displays: Semantic change operations and affected model elements
- **UI-2:** Provide a user interface to browse model versions.
  - Data source: Git version history
  - Extension point: `changevisualization.VersionBrowserService`
  - Displays: Version graph, branches, semantic change summaries, VSUM metadata
- **UI-3:** Provide a user interface for manual conflict resolution, allowing partial merges by the developer.
  - Extension point: `changevisualization.ConflictResolutionUI`
  - Integration: Automatic when high-priority semantic conflicts are detected
  - Display: Side-by-side semantic diff with fine-grained resolution options

4.2 Planned Approach

The design of native branching and model versioning for the Vitruvius server follows a pragmatic strategy: leverage Git's proven version control infrastructure for branching and history management, while extending it with Vitruvius-specific semantic capabilities for consistency preservation and model-aware conflict resolution wherever they satisfy essential requirements. Vitruvius offers fine-grained change operations, composite propagations, and consistency maintenance, which form a strong basis for representing model evolution without reconstructing differences through external tools.

What is missing, however, are first-class concepts such as branches, commits, and version identifiers, as well as a structured way to store and compare diverging histories. The planned approach therefore introduces three focused subsystems: *branch management*, *delta computation and storage*, and *merge infrastructure*, each designed to integrate naturally with Vitruvius' operation-based architecture while avoiding unnecessary complexity.

The following sections describe these subsystems, motivate the design decisions taken, and reference relevant model versioning literature where they align with established techniques.

4.2.1 Branch Management Design

The branch management subsystem leverages Git's native branching infrastructure to manage parallel model evolution while extending it with Vitruvius-specific metadata and validation. Rather than implementing a custom DAG, the system delegates branch topology, history tracking, and ancestry computations to Git, focusing implementation effort on model-aware operations and consistency preservation. This design follows established version control systems such as Git and Mercurial, which organize revision history as a DAG of commits [3][9]. Model-aware differencing and merging tools like EMF DiffMerge and EMFCompare [1] support the comparison and integration of divergent model states, even though they do not maintain their own history graph. Together, these elements provide a solid foundation for computing merge bases and managing parallel evolution safely.

Branch Lifecycle: Branches are managed through Git's native branching mechanism, with Vitruvius maintaining additional metadata for model-specific concerns. Each Git branch corresponds to a Vitruvius development context, with branch creation, switching, and deletion delegated to Git operations while Vitruvius hooks ensure VSUM consistency is validated at key lifecycle points (pre-commit, post-checkout, post-merge).

Branch Identification and Lookup: Branches are identified using Git's standard branch naming. Vitruvius extends this with metadata stored in **.vitruvius/branches/** directory, associating each Git branch with VSUM-specific information such as view configurations, consistency validation history, and merge conflict records.

Branch Switching: Branch switching is performed via Git checkout, triggering Vitruvius post-checkout hooks that reconstruct the VSUM workspace for the target branch. The hook system loads metamodels, correspondences, and transformations from the checked-out state, rebuilds internal indexes, and validates workspace consistency before allowing development to proceed.

Branch Storage: Instead of maintaining branch metadata in a relational database, the hybrid architecture stores branch references in Git's standard **refs/** structure. Vitruvius-specific metadata (consistency snapshots, validation history, merge conflict records) is stored in the **.vitruvius/** directory as file-based artifacts versioned alongside metamodels. This eliminates the need for dual synchronization between Git state and database state.

Branch Topology and Merging: Git's internal commit graph preserves parent-child relationships and the branch DAG. When performing three-way merges, it relies on the merge-base algorithm to compute the lowest common ancestor (LCA), allowing it to efficiently cope with intricate branching structures. Vitruvius augments Git's merge behavior via custom merge drivers that intercept non-line-based files, applying semantic merge logic while leaving the overall structural merge coordination to Git.

Overall, the DAG-based branching strategy integrates naturally with Vitruvius' operation-based evolution model. Branches are treated as references to sequences of deltas rather than full model replicas, fulfilling all core requirements for creation, navigation, identification, storage, and merge preparation while keeping implementation overhead low.

4.2.2 Delta Computation and Storage Concept

The delta computation and storage subsystem adopts a hybrid architecture that delegates file-level versioning to Git while adding Vitruvius-specific semantic change tracking. This approach leverages Git's mature storage, compression, and distribution capabilities while preserving fine-grained operation tracking needed for consistency preservation and semantic conflict detection.

Two-Layer Architecture: The subsystem operates on two complementary layers:

- **Git Storage Layer:** Metamodels
- **Semantic Change Layer:** Vitruvius extends Git commits with semantic changelogs stored in .vitruvius/changelogs/<commit-sha>.changelog. These changelogs capture operation-level changes that are essential for semantic diff computation, conflict detection, and consistency validation but not captured by text-based Git diffs.

Change Recording (Operation-Based): Vitruvius continues to use its ChangeRecorder to capture fine-grained EChange operations during model editing. These operations are aggregated into semantic changelogs during pre-commit hooks, providing structured change representation that complements Git's line-based diffs. The dual representation enables both human-readable evolution tracking and precise semantic merge operations.

State-Based Comparison for Delta Computation: Computing deltas between arbitrary branch states remains necessary for three-way merges. The subsystem extends StateBasedChangeResolutionStrategy to load metamodel states from Git branches and compute structural differences using EMF Compare. This functionality is invoked during merge operations to identify conflicting changes that cannot be detected through text-based comparison alone.

Hybrid Storage Architecture: Rather than implementing a custom content-addressed storage system, the architecture leverages Git's proven object database:

- **Version Objects:** Git presents commits as complete snapshots while internally optimizing storage via delta compression. Git's internal compression and delta encoding eliminate the need for custom snapshot management.
- **Increment Deltas:** Git stores file changes efficiently using packfiles and delta compression. Vitruvius complements this with semantic changelogs that preserve operation causality and ordering for deterministic replay and rollback.
- **Branch Metadata:** Git's refs/ structure maintains branch pointers, while .vitruvius/ and related metadata files track VSUM-specific configuration. This metadata is versioned alongside metamodels, ensuring consistency across branch switches.

Developer-Defined Versions: While Git commits provide fine-grained version history, developers may mark significant milestones as versions with semantic identifiers. These versions are implemented as Git tags referencing specific commits, with additional metadata stored in .vitruvius/versions/ describing the VSUM state, validation status, and associated semantic changes.

```
vitruvius-project/
|– .git/                  # git repository
|    – hooks/             # integration points
|    |   |– pre-commit    # consistency validation
|    |   |– post-checkout # workspace rebuild on branch switch
|    |   |– post-merge    # semantic merge validation
|    |
|    |– refs/             # branch and tag pointers
|    |– objects/          # versioned model content
|
|– .gitattributes         # custom merge drivers
|
|– .vitruvius/            # vitruvius semantic layer
|    |– branches/         # branch metadata
|    |– changelogs/       # operation-based deltas (EChange)
|    |– merges/           # merge metadata and conflicts
|    |– versions/         # version tags and snapshots
|– metamodels/            # versioned EMF metamodels
```

Figure 4.1: Hybrid storage architecture

Performance Optimizations:

- Git's packfile compression minimizes storage overhead for long branch histories without requiring explicit snapshot strategies.
- Semantic changelogs are loaded lazily only when needed for merge operations or semantic diff visualization.
- Git's index enables efficient detection of working directory changes without full model comparison.

This hybrid approach eliminates the complexity of maintaining a custom version control system while preserving Vitruvius's operation-based change semantics. By delegating storage, compression, and distribution to Git, the implementation focuses exclusively on model-aware operations that Git cannot provide: semantic diff computation, consistency validation, and intelligent conflict resolution.

4.2.3 Merge Infrastructure

The merge subsystem reconciles diverging branch histories through three-way merge operations, combining change sequences from source and target branches while detecting and resolving conflicts. This subsystem integrates with the delta computation layer and ensures that Vitruvius's consistency-preservation rules remain satisfied throughout the merge process.

Integration with Git Merge: The merge subsystem integrates with Git's merge workflow through custom merge drivers registered for Vitruvius-specific context. When Git detects conflicts in, for example, .ecore files, it delegates to Vitruvius-specific merge drivers that perform semantic three-way merges instead of line-based text merges. Non-conflicting files are merged by Git using its standard algorithms. This division of responsibility ensures that Git handles structural merge orchestration while Vitruvius provides domain-specific conflict detection and resolution.

Three-Way Merge Algorithm: The merge procedure builds on Git's standard three-way merge mechanism and augments it with Vitruvius's semantic merging features. When a developer runs ‘git merge‘, the following steps are executed: First, Git determines the merge base (LCA) using its internal commit graph algorithms. Second, Git attempts to merge all files; for Vitruvius-specific artifacts such as ‘.ecore‘ or ‘.reactions‘ files, custom merge drivers are triggered. Third, the merge driver loads the three variants (base, ours, theirs) as EMF models and derives structural differences via EMFCompare. Fourth, it categorizes conflicts and automatically applies all non-conflicting changes. Fifth, any conflicting modifications are annotated with semantic conflict markers and exposed to the developer for manual resolution. Sixth, post-merge hooks check VSUM consistency before the merge is finalized. In the last step, the developer completes the process by committing the resolved state.

Conflict Classification and Resolution: Detected conflicts are classified by priority (HIGH, MEDIUM, LOW) based on factors such as the severity of the conflict type, the importance of affected elements, and the number of dependent elements. High-priority conflicts (e.g., DELETE-UPDATE, consistency violations) require manual resolution. Medium-priority conflicts (e.g., UPDATE-UPDATE with different values) receive suggested resolutions that developers can accept or override. Low-priority conflicts are resolved automatically using predefined heuristics.

Resolution strategies are pluggable, inspired by EML's concept of domain-specific merge rules. Automatic resolution applies heuristics for trivial cases (e.g., accepting identical changes from both branches). Semi-automatic resolution presents conflicts with recommendations based on change patterns and timestamps. Manual resolution provides a user interface displaying conflicting changes side-by-side with options to select one version, merge manually, or defer resolution.

Merge History and Integrity: Each completed merge creates a merge commit with dual-parent references linking both the source and target branches. This maintains complete lineage in the branch DAG, enabling accurate ancestry queries for future merges and supporting audit trails. Merge commits are distinguished from regular commits to ensure the branch topology correctly represents the merge structure. The merge history is stored in the semantic change layer alongside branch and version metadata, ensuring that the integrity of the overall model history is preserved throughout parallel development and integration.

Overall, the merge infrastructure combines proven three-way merge algorithms with operation-based conflict detection and Vitruvius-specific semantic validation. By building on AMOR's conflict patterns, EMF Diff/Merge's workflow structure, and EML's pluggable resolution concepts, the subsystem provides robust merging capabilities tailored to Vitruvius's consistency-preservation architecture while maintaining compatibility with the existing change tracking and propagation mechanisms.

4.2.4 Integration Architecture: Git and Vitruvius Interaction

The hybrid architecture establishes clear boundaries between Git's responsibilities and Vitruvius's domain-specific logic through three primary integration mechanisms:

Git Hooks: Git's lifecycle hooks (pre-commit, post-checkout, post-merge) serve as integration points where Vitruvius intercepts Git operations to perform model-specific validation and workspace management. Pre-commit hooks validate VSUM consistency and generate semantic changelogs. Post-checkout hooks rebuild the VSUM workspace for the target branch. Post-merge hooks validate the merged state and detect semantic conflicts that survived text-based merge.

Custom Merge Drivers: Vitruvius registers custom merge drivers for non-line-based files through Git's .gitattributes mechanism. When Git detects conflicts in these files, it invokes the appropriate merge driver instead of performing line-based text merge. Merge drivers load files as EMF models, perform semantic three-way merge, and write results with semantic conflict markers if necessary.

Semantic Metadata Layer: The .vitruvius/ directory stores model-specific metadata (changelogs, consistency snapshots, validation history) that is versioned alongside metamodels. This metadata is accessible to hooks and merge drivers, enabling semantic operations while remaining transparent to standard Git operations. Developers using standard Git commands benefit from Vitruvius enhancements automatically without requiring specialized tools. This architecture achieves a clean separation: Git manages version topology, history, and distribution; Vitruvius ensures semantic correctness and consistency. The integration is sufficiently loose that Vitruvius enhancements are optional-projects can use Git alone for basic versioning, adopting Vitruvius hooks and merge drivers when consistency preservation becomes critical.

4.3 Expected Workflow

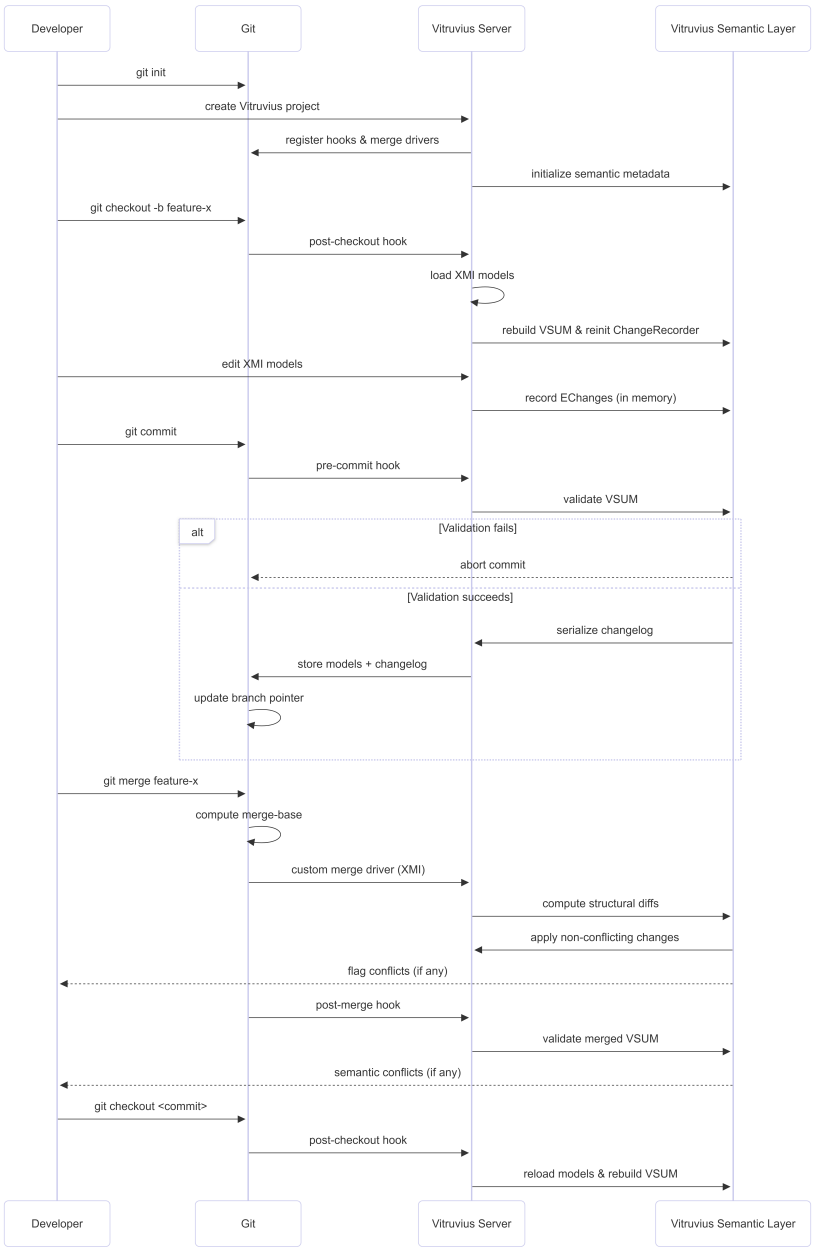

Figure 4.2: Git-integrated Vitruvius workflow, showing interactions between Developer, Git, Vitruvius Server, and Semantic Layer during branching, editing, committing, and merging.

5 Organisation

To present the planned structure and workflow of the thesis, this chapter introduces the main phases of conception, implementation, and validation, and explains how an incremental development strategy, combined with systematic testing, ensures that the branching and versioning infrastructure is implemented, integrated, and verified effectively.

5.1 Phases

The thesis is structured into four main phases: conception design, implementation, and validation. While these phases are presented sequentially, feedback loops are expected, particularly between design and implementation.

5.1.1 Conception and Design

In this phase, a concrete design for server-side branching support in Vitruvius is developed. This includes the design of the branch management subsystem, the delta computation and storage strategy, and the merge infrastructure. Design decisions are justified based on the requirements and the limitations of existing approaches identified in the related work. Where appropriate, optional extensions and reuse of existing frameworks are evaluated with respect to feasibility and implementation effort.

5.1.2 Implementation Strategy

The implementation follows an iterative, Git-centered development strategy. Instead of introducing a custom version control mechanism, Git is reused as the authoritative backend for branching, version history, and storage wherever possible. Vitruvius is extended incrementally with model-specific semantics only where Git does not provide sufficient support. All iterations initially focus on XMI-based models to limit complexity and enable systematic validation.

Each iteration delivers a runnable and testable system that integrates branching, storage, and validation, with increasing semantic depth.

Iteration 1: Git-Based Branching and Storage Integration The first iteration focuses on fully integrating Git into the Vitruvius Server. Branch creation, switching, deletion, and lookup are delegated to Git operations, while branch topology and ancestry are derived from Git's commit DAG. Model snapshots are stored as XMI files in Git commits, relying on Git's object database and packfile-based delta compression for efficient storage.

Integration is realized via Git hooks:

- **post-checkout:** reload XMI models, rebuild the VSUM workspace, reinitialize the ChangeRecorder
- **pre-commit:** validate VSUM consistency and generate semantic changelogs
- **post-merge:** validate the merged state and detect semantic conflicts

At this stage, Git's standard three-way merge is used without custom merge drivers and Vitruvius performs semantic validation after merge execution.

Iteration 2: Semantic Delta Storage and Replay The second iteration extends the system with semantic change persistence. Operation-based EChange sequences recorded by Vitruvius are serialized as XMI-based changelogs and associated with Git commits. This enables semantic inspection, replay, and rollback while preserving Git as the primary storage mechanism.

Iteration 3: Model-Aware Merge Support This iteration introduces custom merge drivers for XMI-based models. Git continues to compute merge bases and orchestrate merges, while Vitruvius performs semantic three-way merges using EMF Compare. Non-conflicting changes are applied automatically while conflicting changes are detected and classified.

Iteration 4: Conflict Resolution and Extensions The final iteration implements conflict resolution strategies and optional extensions such as conflict prioritization, limited automation, and improved diagnostics. Depending on available time, support for additional artifact types or advanced conflict patterns may be explored.

5.1.3 Validation and Evaluation

Validation is performed continuously across all iterations. Each iteration includes unit and integration tests covering branch operations, storage, validation hooks, and merge scenarios. Correctness is assessed by verifying VSUM consistency, reproducibility of model states, and preservation of branch history. If time permits, the approach is evaluated using an existing Vitruvius case study to demonstrate practical applicability.

5.2 Timeline

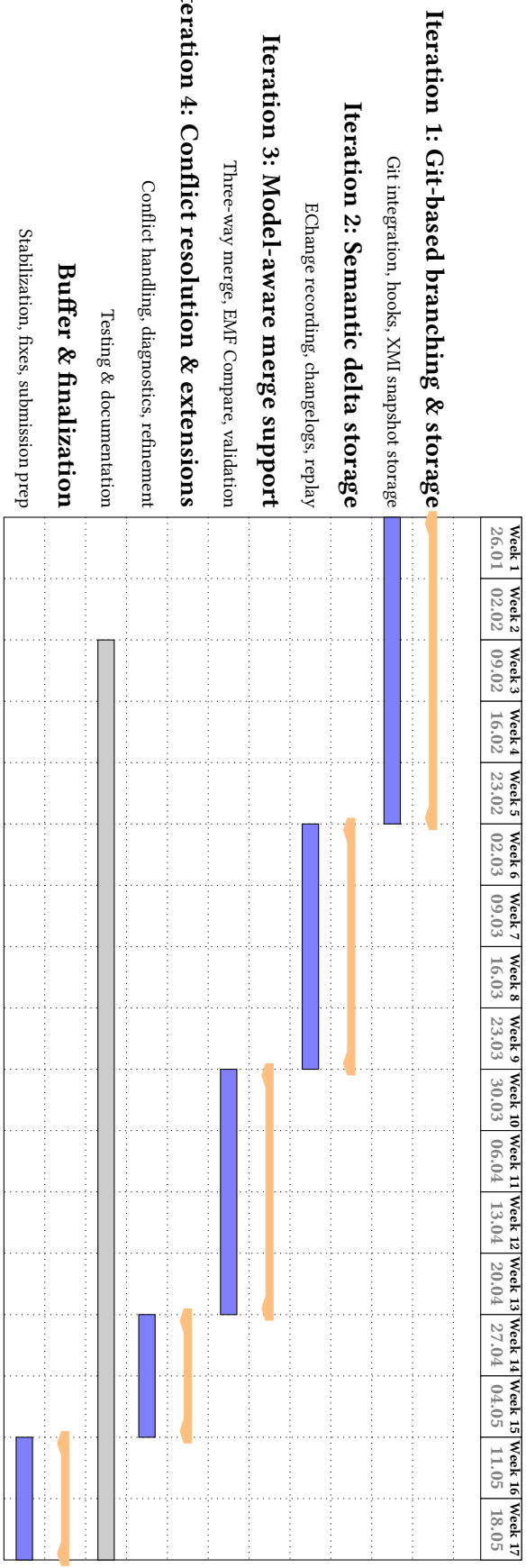

Figure 5.1: Iterative implementation timeline from 26 January to 26 May

Bibliography

[1] Maik Appeldorn, Dilshodbek Kuryazov, and Andreas Winter. "Delta-driven collaborative modeling." In: *MoDELS (Workshops)*. 2018, pp. 293–302.

[2] Colin Atkinson and Thomas Kuhne. "Model-driven development: a metamodeling foundation". In: *IEEE software* 20.5 (2003), pp. 36–41.

[3] Petr Baudiš. "Current concepts in version control systems". In: *arXiv preprint arXiv:1405.3496* (2014).

[4] Petra Brosch et al. "An introduction to model versioning". In: *International school on formal methods for the design of computer, communication and software systems*. Springer, 2012, pp. 336–398.

[5] Cédric Brun and Alfonso Pierantonio. "Model differences in the eclipse modeling framework". In: *UPGRADE, The European Journal for the Informatics Professional* 9.2 (2008), pp. 29–34.

[6] Hugo Bruneliere et al. "A feature-based survey of model view approaches". In: *Software & Systems Modeling* 18.3 (2019), pp. 1931–1952.

[7] Antonio Cicchetti, Davide Di Ruscio, and Alfonso Pierantonio. "Managing model conflicts in distributed development". In: *International Conference on Model Driven Engineering Languages and Systems*. Springer. 2008, pp. 311–325.

[8] Mary Rose Cook. *Git From the inside-out*. URL: `https://maryrosecook.com/blog/post/git-from-the-inside-out`.

[9] N Deepa et al. "An analysis on version control systems". In: *2020 international conference on emerging trends in information technology and engineering (ic-ETITE)*. IEEE. 2020, pp. 1–9.

[10] Xiao He et al. "Accelerating similarity-based model matching with subtree equivalence". In: *Information and Software Technology* (2025), p. 107879.

[11] Benedikt Jutz and Thomas Weber. "Scalability of Consistency Preservation with Vitruvius". In: *Softwaretechnik-Trends Band 44, Heft 4*. Gesellschaft für Informatik e.V., 2024.

[12] Heiko Klare et al. "Enabling consistency in view-based system development—the vitruvius approach". In: *Journal of Systems and Software* 171 (2021), p. 110815.

[13] Maximilian Koegel and Jonas Helming. "EMFStore: a model repository for EMF models". In: *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering-Volume 2*. 2010, pp. 307–308.

[14] Dimitrios S Kolovos, Richard F Paige, and Fiona AC Polack. "Merging models with the epsilon merging language (eml)". In: *International conference on model driven engineering languages and systems*. Springer. 2006, pp. 215–229.

[15] Richard F Paige, Nicholas Matragkas, and Louis M Rose. "Evolving models in model-driven engineering: State-of-the-art and future challenges". In: *Journal of Systems and Software* 111 (2016), pp. 272–280.

[16] Jakob Pietron. "Enhancing collaborative modeling". In: *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*. 2020, pp. 1–6.

[17] Daniel Seybold et al. "Experiences of models@ run-time with EMF and CDO". In: *Proceedings of the 2016 ACM SIGPLAN International Conference on Software Language Engineering*. 2016, pp. 46–56.

[18] Mohammadreza Sharbaf and Bahman Zamani. "Configurable three-way model merging". In: *Software: Practice and Experience* 50.8 (2020), pp. 1565–1599.

[19] EMFCompare DEV Team. *EMFCompare Homepage*. URL: `https://eclipse.dev/emf/compare/`.

[20] Git Team. *Git Book*. URL: `https://git-scm.com/book/en/v2`.